

ĐẶC TẢ GIAO THỨC

Distributed Cache System — Nhóm 05 — Lớp 23DTHA5 — Lập trình mạng máy tính

1. Mục đích và nguyên tắc chung

Tài liệu này là bản đặc tả kỹ thuật dùng chung cho cả 5 module, để module nào cũng nói cùng một "ngôn ngữ" khi giao tiếp qua mạng. Bất kỳ thay đổi nào ở đây đều phải được cả nhóm đồng ý trước khi code, vì sửa sau khi đã code sẽ ảnh hưởng dây chuyền tới các module khác.

Nguyên tắc chung:

- Giao thức là văn bản thuần (text-based), mã hóa UTF-8, mỗi message kết thúc bằng ký tự xuống dòng \n.
- Vận chuyển qua TCP Socket (java.net.Socket / java.net.ServerSocket), không dùng UDP.
- Dấu phân cách trường trong message là ký tự | (pipe). Giá trị (value) không được chứa ký tự | hoặc \n ở bản cơ bản.
- Mỗi kết nối TCP xử lý đúng 1 request rồi đóng (connection-per-request), trừ khi module liên quan quyết định dùng kết nối thường trực — nếu vậy phải ghi rõ trong mục 8.

2. Định dạng message từ Client → Cache Server

Lệnh	Cú pháp	Ví dụ
PUT	PUT <key> <value>\n	PUT username Duy\n
GET	GET <key>\n	GET username\n
DELETE	DELETE <key>\n	DELETE username\n

Quy ước trường:

- key: chuỗi không rỗng, không chứa | hay \n, tối đa 256 ký tự.
- value: chuỗi bất kỳ trừ | và \n, tối đa 4096 ký tự (giới hạn này có thể điều chỉnh, ghi rõ trong code bằng hằng số MAX_VALUE_LENGTH).
- Tên lệnh (PUT, GET, DELETE) luôn viết HOA, không phân biệt hoa/thường khi parse để tránh lỗi vật.

Ví dụ dòng gửi thực tế qua socket (kết thúc bằng \n, không gửi kèm ký tự thừa):

```
PUT|session:abc123|{"userId":42}
```

3. Định dạng response từ Cache Server → Client

Kết quả	Cú pháp	Áp dụng cho lệnh
Thành công (ghi/xóa)	OK\n	PUT, DELETE
Thành công (đọc)	VALUE <value>\n	GET
Không tìm thấy key	NOTFOUND\n	GET, DELETE
Lỗi	ERROR <mã lỗi> <mô tả>\n	Tất cả

Bảng mã lỗi chuẩn (cả nhóm dùng chung, không được tự đặt mã mới nếu chưa thống nhất):

Mã lỗi	Ý nghĩa	Ví dụ đầy đủ
E001	Cú pháp lệnh không hợp lệ	ERROR E001 Invalid command syntax
E002	Key rỗng hoặc quá dài	ERROR E002 Key exceeds max length
E003	Value quá dài	ERROR E003 Value exceeds max length
E004	Server quá tải / từ chối kết nối	ERROR E004 Server overloaded

E005	Lỗi nội bộ server (exception không xác định)	ERROR E005 Internal server error
E006	Server đang đồng bộ (replication), tạm từ chối ghi	ERROR E006 Server syncing, retry later

Client (Module 2) dựa vào cột mã lỗi để quyết định có nên failover sang server khác hay không: E004 và E006 nên kích hoạt failover/retry, các mã còn lại là lỗi dữ liệu nên không cần đổi server.

4. Quy ước hashing và cấu hình server

Công thức chọn server (dùng chung, không đổi giữa các module):

```
serverIndex = |hash(key)| mod N
```

Trong đó `hash(key)` là `key.hashCode()` của Java String, `N` là tổng số Cache Server đang cấu hình (không phải số server đang sống — `N` cố định theo file cấu hình để mọi client tính ra cùng một kết quả).

Danh sách server đọc từ file `config.properties` đặt cùng thư mục với Client, format:

```
server.0.host=127.0.0.1
server.0.port=5001
server.1.host=127.0.0.1
server.1.port=5002
server.2.host=127.0.0.1
server.2.port=5003
```

Chỉ số `server.<index>` phải đánh từ 0 liên tục, không được nhảy số, vì `index` này chính là `serverIndex` dùng trong công thức hashing ở trên. Server dự phòng (dùng cho replication ở mục 5) là server tại $(\text{serverIndex} + 1) \bmod N$.

5. Giao thức Replication (Server ↔ Server)

Đây là kênh giao tiếp riêng giữa các Cache Server với nhau, không phải giữa Client và Server. Khi Server chính (`serverIndex`) nhận được lệnh PUT/DELETE thành công, nó gửi tiếp lệnh sau tới Server dự phòng $((\text{serverIndex} + 1) \bmod N)$ trên một port riêng dành cho replication (khác port phục vụ client):

Lệnh	Cú pháp	Mô tả
REPLICATE_PUT	REPLICATE PUT <key> <value> <timestamp>\n	Đồng bộ một key vừa được ghi
REPLICATE_DEL	REPLICATE DELETE <key> <timestamp>\n	Đồng bộ một key vừa bị xóa

`timestamp` là `System.currentTimeMillis()` tại thời điểm ghi trên server chính, dùng để giải quyết xung đột: khi server dự phòng phục hồi sau sự cố và nhận 2 bản ghi khác nhau cho cùng 1 key, bản ghi có `timestamp` lớn hơn (mới hơn) được giữ lại.

Server dự phòng phản hồi `OK\n` sau khi ghi thành công vào kho dữ liệu của nó. Server chính **không** chờ phản hồi này trước khi trả OK về cho Client (replication bất đồng bộ — asynchronous), để không làm chậm đường đi chính. Nếu gửi replication thất bại (server dự phòng đang chết), server chính ghi log cảnh báo và bỏ qua — không retry vô hạn.

6. Giao thức Heartbeat (Cache Server ↔ Registry)

Registry (Module 4) là 1 tiến trình Java riêng, lắng nghe trên 1 port cố định (quy ước: 6000). Mỗi Cache Server, sau khi khởi động, mở 1 thread nền gửi heartbeat định kỳ mỗi 3 giây:

Lệnh	Cú pháp	Hướng gửi
HEARTBEAT	HEARTBEAT <serverIndex> <host> <port>\n	Cache Server → Registry

ACK	ACK\n	Registry → Cache Server
-----	-------	-------------------------

Registry lưu trạng thái `serverIndex` → `lastHeartbeatTimestamp` trong `ConcurrentHashMap`. Nếu một server không gửi heartbeat quá **9 giây** (trễ cứng dồn = 3 lần chu kỳ), Registry coi server đó là đã chết (DOWN).

Client (Module 2) truy vấn trạng thái này trước khi failover:

Lệnh	Cú pháp	Phản hồi
STATUS	STATUS <serverIndex>\n	UP\n hoặc DOWN\n
STATS	STATS\n	STATS <serverIndex>:<UP DOWN>:<số key>:<số request>;... (dùng cho GUI dashboard Module 5)

7. Cấu trúc package Java thống nhất

Cả nhóm dùng chung 1 repo Maven, mỗi thành viên chỉ code trong package của module mình để hạn chế xung đột khi merge Git:

<code>com.nhom05.cache.common</code>	(dùng chung: hằng số mã lỗi, parser message, tiện ích hashing)
<code>com.nhom05.cache.server</code>	(Module 1 - Core Cache Server / KV engine)
<code>com.nhom05.cache.client</code>	(Module 2 - Client + Hashing Router + Failover)
<code>com.nhom05.cache.replication</code>	(Module 3 - Replication & Consistency)
<code>com.nhom05.cache.registry</code>	(Module 4 - Monitoring / Registry / Health Check)
<code>com.nhom05.cache.gui</code>	(Module 5 - GUI, dashboard)

Package common chứa các class dùng chung cho toàn bộ hệ thống, do nhóm trưởng quản lý/review trước khi merge vì thay đổi ở đây ảnh hưởng tới tất cả module khác:

- `ErrorCode.java` — enum chứa các mã lỗi ở mục 3
- `ProtocolParser.java` — parse/build chuỗi message theo đúng cú pháp ở mục 2, 5, 6
- `HashUtil.java` — hàm `computeServerIndex(String key, int n)` theo công thức ở mục 4
- `ServerConfig.java` — đọc `config.properties`

8. Lưu ý triển khai

- Timeout: Client đặt `Socket.setSoTimeout(2000)` (2 giây) cho mọi kết nối tới Cache Server. Quá thời gian này mà không có phản hồi → coi như server không phản hồi, kích hoạt failover.
- Retry/Failover: tối đa thử 1 server dự phòng $((serverIndex + 1) \bmod N)$, không lặp lại toàn bộ danh sách để tránh treo client khi cả hệ thống sập. Nếu cả 2 đều không phản hồi, client trả lỗi E004 cho người dùng/GUI.
- Thread-safety: mọi thao tác đọc/ghi trên kho dữ liệu chính phải dùng `ConcurrentHashMap`, không dùng `HashMap` thuần + `synchronized` thủ công trừ khi cần thao tác compound (vd: check-then-act) thì bọc trong `synchronized` hoặc dùng `computeIfAbsent/compute`.
- Encoding: đọc/ghi socket stream bằng `BufferedReader/PrintWriter` với charset UTF-8 tường minh (tránh lỗi hiển thị tiếng Việt có dấu trong value).
- Thứ tự hoàn thiện: Module 1 và 2 phải xong trước (có thể test PUT/GET/DELETE qua telnet) rồi Module 3, 4 mới gắn vào được; Module 5 (GUI) phụ thuộc vào cả 4 module còn lại nên làm song song phiên bản giả lập (mock) rồi nối dần vào backend thật.